

Processes - Exam Questions and Answers

Question 1

Write the code of a program that can be used to make a copy of a certain file, specified by the user. This program should be run by two threads T1 and T2 belonging to two different processes P1 and P2. T1 belongs to P1, and T2 belongs to P2. T1 is the thread responsible for asking which file to copy, and the file name of the copy, while T2 is the thread responsible for making the actual copy.

Question 2

Why does the OS define a “terminated” state for a process and not simply end the process? Provide an example of when this is useful.

Question 3

Indicate which connections are wrong and which ones are correct in the following figure. For each wrong or missing connection, explain why you think the connection is either wrong or missing.

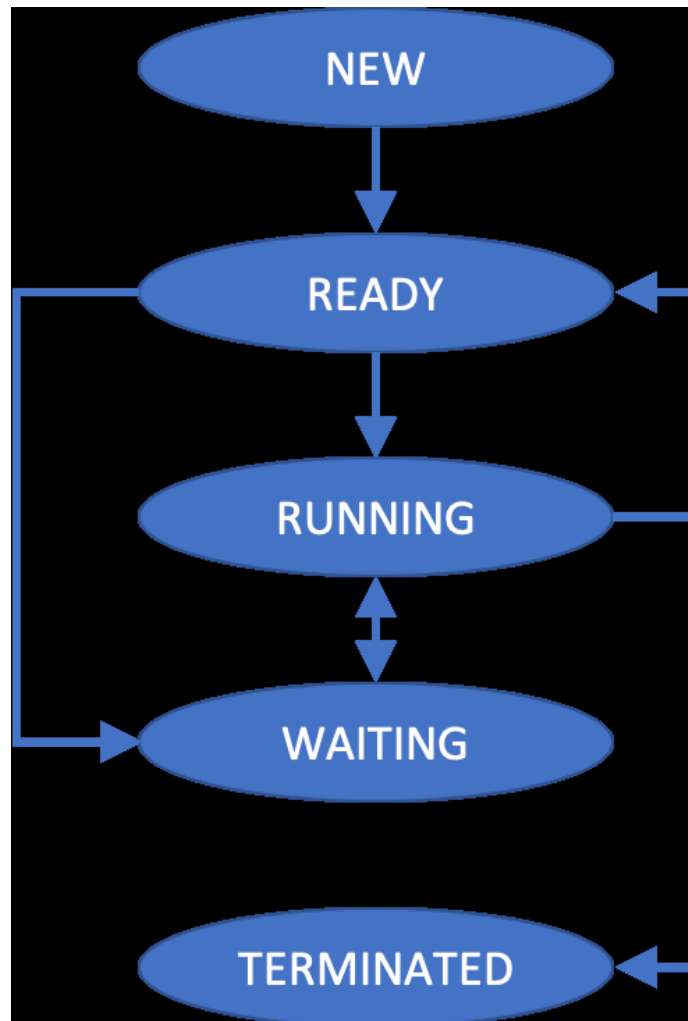


Figure 1: Process-state diagram

Question 4

Is the following code going to print **Greetings** or not? Explain why. The listing contains only the relevant portions of code, so the answer does not need to take minor inconsistencies, such as missing include statements, into account.

```
int main(void)
{
    char write_msg[BUFFER_SIZE] = "Greetings";
    char read_msg[BUFFER_SIZE];
    pid_t pid;
    int fd[2];

    /* now fork a child process */
    pid = fork();

    /* create the pipe */
    if (pipe(fd) == -1) {
        fprintf(stderr, "Pipe failed");
        return 1;
    }

    if (pid < 0) {
        fprintf(stderr, "Fork failed");
        return 1;
    }

    if (pid > 0) { /* parent process */
        /* close the unused end of the pipe */
        close(fd[READ_END]);
        /* write to the pipe */
        write(fd[WRITE_END], write_msg, strlen(write_msg) + 1);
        /* close the write end of the pipe */
        close(fd[WRITE_END]);
    }
    else { /* child process */
        /* close the unused end of the pipe */
        close(fd[WRITE_END]);
        /* read from the pipe */
        read(fd[READ_END], read_msg, BUFFER_SIZE);
        printf("child read %s\n", read_msg);
        /* close the write end of the pipe */
        close(fd[READ_END]);
    }

    return 0;
}
```